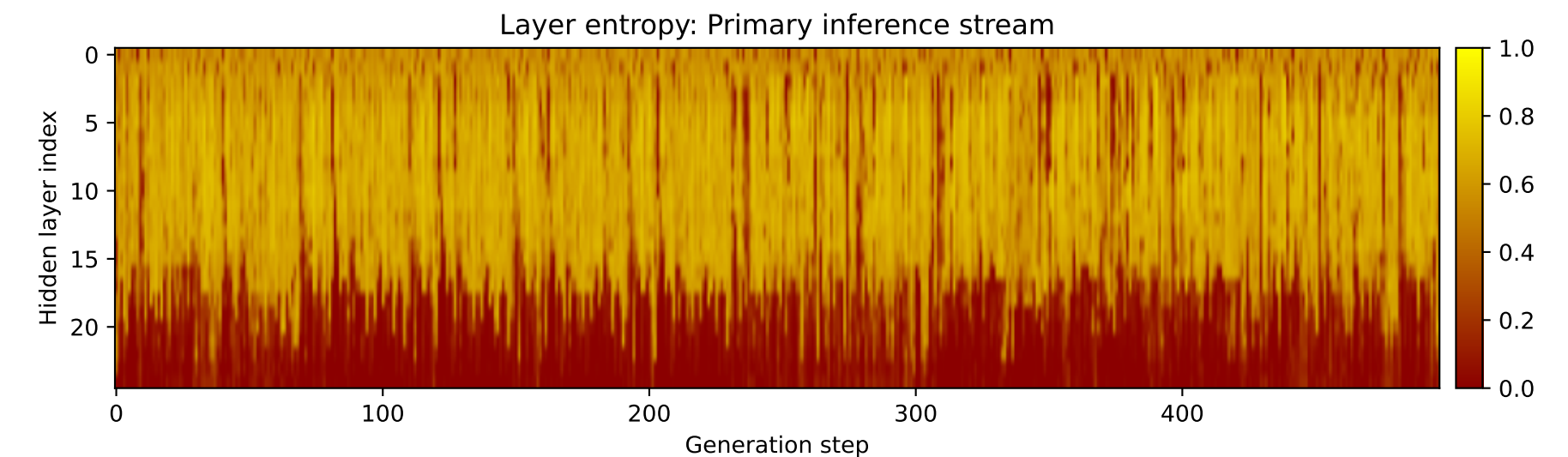
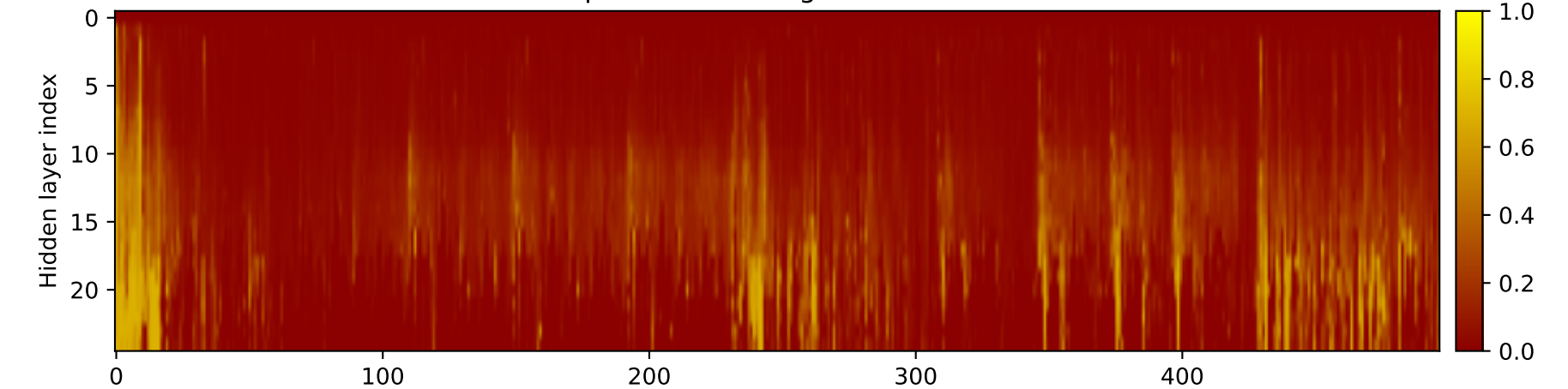
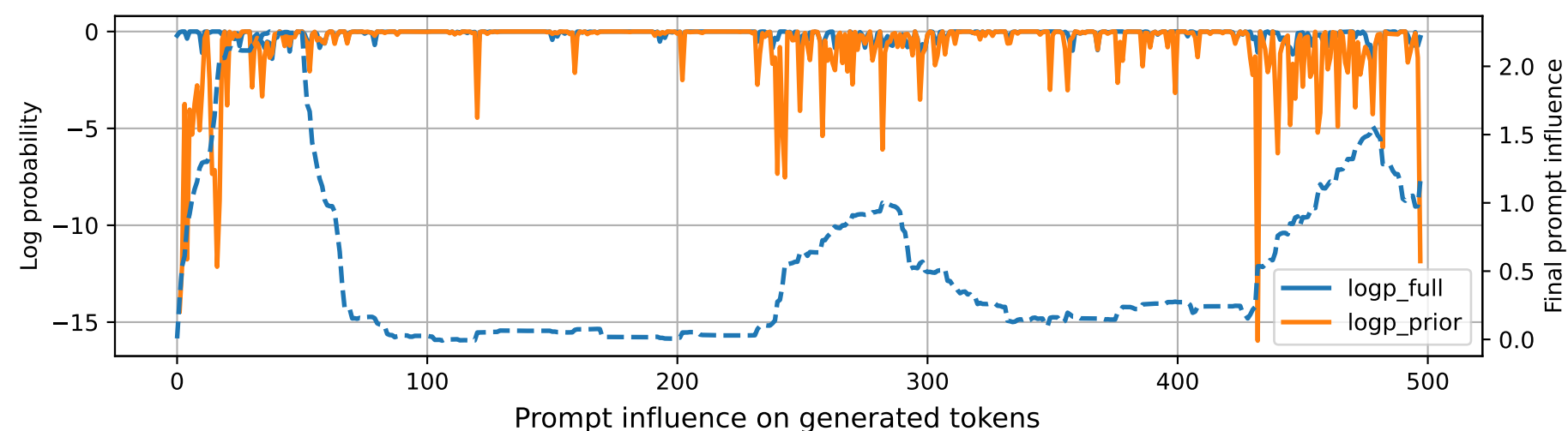


Prompt 1



PROMPT:  
Task ID: PYC083 Return ONLY valid Python code. Do not use  
Markdown fences. Do not include prose,  
explanations, examples, print statements, or tests. Your code  
must contain this exact check-token line: #  
CHECK\_TOKEN: PYC083 Your code must contain this required symbol:  
def validate\_user\_records Implement this  
task: write function validate\_user\_records(rows) returning errors  
list with row index and message.. Start  
your answer with the check-token line, then provide the Python  
implementation.

MODEL RESPONSE:  
# CHECK\_TOKEN: PYC083 ```python def validate\_user\_records(rows):  
 errors = [] for row in rows:  
# Check if the row has a valid email address if not  
row['email']: errors.append(f"Row  
{row.index} has an invalid email address: {row['email']}")  
 # Check if the row has a valid phone  
number if not row['phone']:  
errors.append(f"Row {row.index} has an invalid phone number:  
{row['phone']}") # Check if the row has a valid password  
 if not row['password']:  
errors.append(f"Row {row.index} has an invalid password:  
{row['password']}") # Check if the row has a  
valid date of birth if not row['dob']:  
errors.append(f"Row {row.index} has an invalid date  
of birth: {row['dob']}") # Check if the row has a valid  
address if not row['address']:  
errors.append(f"Row {row.index} has an invalid address:  
{row['address']}") return errors ``` This  
function `validate\_user\_records` takes a list of rows as input  
and returns a list of errors with the row index  
and message for each row that does not meet the validation  
criteria. The validation criteria include: 1.  
Email address: The email address should be a valid email address  
according to the standard format (e.g.,  
`user@example.com`). 2. Phone number: The phone number should be  
a valid phone number according to the  
standard format (e.g., `123-456-7890`). 3. Password: The password  
should be a valid password according to the  
standard format (e.g., `password123`). 4. Date of birth: The date  
of birth should be a valid date in the  
format `YYYY-MM-DD`. 5. Address: The address should be a valid  
address according to the standard format (e.g.,  
`123 Main St, Anytown, USA`). The function iterates through each  
row in the input list and checks if the  
corresponding column has a valid value. If any of the validation  
criteria fail, an error message is appended  
to the `errors` list with the row index and the corresponding  
validation error message. Finally, the function  
returns the `errors` list containing the validation errors.